

AUTO AI BUILDER

Project Handoff & Technical Onboarding

Destinatário: Pablo - Software Engineer

Product Owner / Domain Specialist: Danilo Prates Coelho - Civil Engineer

Documento: Technical Project Handoff

Data de referência: 01/10/2026

Status do projeto: Protótipo funcional parcial / desenvolvimento ativo

Plataforma principal: Windows

Ecossistema alvo: AutoCAD + AltoQi Builder + eventualmente TQS

Nome do produto: Auto AI Builder

Objetivo deste documento

Permitir que um novo desenvolvedor entre no Auto AI Builder entendendo o problema real de engenharia, o histórico dos experimentos, as decisões arquiteturais, o que já foi validado, o que continua em aberto e exatamente de onde o desenvolvimento deve retomar.

Como ler este documento

Este handoff não é um resumo executivo superficial. Ele registra o conhecimento acumulado durante o desenvolvimento e os experimentos realizados com AutoCAD, AltoQi Builder, Computer Use, Codex e, mais recentemente, o MCP do AltoQi Builder.

Ao longo do texto, os itens são classificados implicitamente ou explicitamente em três estados:

- **[VALIDADO]**: comportamento ou conclusão confirmada em experimento suficiente para servir como referência atual.
- **[EM DESENVOLVIMENTO]**: hipótese, arquitetura ou funcionalidade promissora, mas ainda sem validação completa.
- **[NAO REPETIR]**: abordagem que mostrou fragilidade, perda de informação ou inconsistência e não deve ser retomada sem uma justificativa nova.

Números de versões antigas são preservados porque ajudam a reconstruir a evolução do classificador. Quando houver contagens diferentes entre snapshots históricos, o documento indica isso explicitamente.

Mapa de navegação

Faixa	Conteúdo principal
Seções 1-12	Problema, visão, princípios, IA, stack, arquitetura, modelo e interface
Seções 13-36	Experimentos de máscara, classificação, layers, baseline e regressão
Seções 37-58	Lançamento no Builder, geometria, offsets e Computer Use
Seções 59-72	MCP do Builder, AutoCAD, persistência, logs e confiança
Seções 73-98	Conduitos, circuitos, disciplinas, MVP, status e produto
Seções 99-123	Estado atual, riscos, colaboração, spikes, roadmap e dados a preservar
Seções 124-128	Glossário, mental model, arquitetura futura, conclusão e primeiro dia
Apêndices A-G	Baselines, casos geométricos, checklists e estrutura recomendada

1. Resumo executivo

O Auto AI Builder nasceu de um problema muito concreto do trabalho diário de projetos complementares: uma parte relevante do tempo não está na tomada de decisão de engenharia, mas em tarefas repetitivas de interpretação de desenhos, preparação de máscaras, classificação de elementos, lançamentos, posicionamentos, associações, criação de circuitos, lançamento de tubulações/conduitos e validação operacional dentro dos softwares.

Danilo trabalha principalmente com:

- projetos elétricos;
- projetos hidrossanitários;
- projetos estruturais;
- AutoCAD;
- AltoQi Builder;
- TQS.

A visão do Auto AI Builder é criar uma plataforma desktop capaz de transformar uma planta arquitetônica ou projeto-base em um **modelo semântico compreendido pelo software**, aplicar regras e posteriormente executar ou auxiliar os lançamentos necessários nos programas de engenharia.

A sequência conceitual do produto é:

Importar -> Mascara -> Identificacao -> Regras -> Proposicao -> Lancamento -> Validacao -> Relatorios

O princípio fundamental é:

Automatizar trabalho repetitivo sem remover do engenheiro a responsabilidade técnica e o poder de revisão.

O produto deve funcionar inicialmente como um **copiloto técnico extremamente competente**, e não como um sistema autônomo que altera projetos sem supervisão.

2. Problema de negócio

Em projetos complementares, frequentemente existe uma arquitetura recebida em DWG contendo milhares de entidades, entre elas:

- paredes;
- portas;
- janelas;
- cotas;
- textos;
- mobiliários;
- símbolos;
- legendas;
- pontos elétricos;
- pontos hidráulicos;
- detalhes gráficos;
- vistas;
- elementos duplicados;
- informações sem relevância para o projeto complementar.

Antes mesmo de começar o projeto elétrico ou hidrossanitário, o projetista precisa interpretar e organizar esse conteúdo.

Depois disso ainda existem tarefas como:

- identificar pontos;
- interpretar símbolos;
- interpretar textos próximos aos símbolos;
- distinguir tomadas 10 A / 20 A;
- identificar tensão;
- identificar altura;
- determinar orientação;
- selecionar peças no Builder;
- lançar peças;
- ajustar posição;
- criar circuitos;
- interligar pontos;
- lançar condutos;
- revisar o resultado;
- corrigir erros visuais ou de interpretação.

O projeto pretende transformar grande parte desse processo em um pipeline computacional estruturado.

3. Visão do produto

O Auto AI Builder não deve ser simplesmente um “robô que clica”. A visão evoluiu para uma plataforma com quatro grandes capacidades.

3.1. Entendimento do desenho

O software deve conseguir transformar um DWG em informações como:

- ambientes;
- paredes;
- portas;
- janelas;
- mobiliário relevante;
- pontos elétricos;
- pontos hidráulicos;
- pontos sanitários;
- textos associados;
- orientação;
- posição;
- layer;
- bloco;
- handle;
- rotação;
- atributos;
- relações espaciais;
- confiança da classificação.

3.2. Aplicação de regras

Depois de interpretar o desenho, o sistema deve ser capaz de usar regras de engenharia e de projeto.

Exemplos:

- determinada simbologia representa uma tomada;
- determinada tomada está a determinada altura;
- um texto 20A pertence à tomada mais próxima segundo determinadas condições;
- um texto 220V modifica a especificação daquele ponto;
- duas entidades gráficas representam uma tomada dupla;
- determinado ponto pertence a um ambiente;
- determinados pontos provavelmente pertencem ao mesmo circuito;
- determinado lançamento deve ser revisado pelo usuário.

3.3. Execução

O sistema poderá posteriormente executar ações em:

- AutoCAD;
- AltoQi Builder;
- eventualmente TQS.

A preferência é sempre por integração programática quando disponível. Automação visual deve ser considerada uma alternativa ou complemento, não a primeira escolha quando existir uma API ou integração melhor.

3.4. Supervisão

Nenhuma automação crítica deve ser completamente opaca.

O usuário deve conseguir ver:

- o que foi identificado;
 - por que foi identificado;
 - posição;
 - classificação;
 - confiança;
 - associações;
 - alterações propostas;
 - alterações executadas;
 - erros;
 - itens pendentes de revisão.
-

4. Princípios de engenharia do produto

Estes princípios surgiram repetidamente durante os experimentos e devem ser tratados como requisitos arquiteturais.

4.1. Não destruir o arquivo original

Regra básica:

Nunca trabalhar de forma destrutiva sobre o DWG original.

Workflow ideal:

Original -> Cópia de trabalho -> Análise -> Prévia -> Aprovação -> Resultado

Nunca substituir o arquivo original silenciosamente.

4.2. Automação auditável

Uma ação executada pelo programa deve poder responder:

- o que foi alterado;
- quando;
- onde;
- por qual módulo;
- baseado em qual regra;
- qual era o estado anterior;
- qual o estado posterior.

Por isso existe a ideia de:

- snapshots;
- logs;
- histórico;
- SQLite;
- trilha de auditoria.

4.3. IA não deve executar geometria simples

Operações como:

- distância;
- rotação;
- filtro;
- cálculo de coordenadas;

- busca por handle;
- transformação geométrica;
- interseção;
- proximidade;
- consulta de layer;
- associação determinística;

devem ser feitas pelo software tradicional.

Não há motivo para usar um LLM para calcular coisas que um algoritmo determinístico resolve com precisão.

5. Filosofia de IA

Uma das discussões mais importantes do projeto foi:

“Esse software consegue funcionar sem uma IA funcionando por trás?”

A resposta definida conceitualmente foi:

A base do Auto AI Builder deve funcionar sem IA generativa.

A IA deve ser uma camada de inteligência adicional.

5.1. Núcleo determinístico

Responsável por:

- leitura do DWG;
- geometria;
- layers;
- cores;
- blocos;
- handles;
- textos;
- coordenadas;
- rotações;
- associação espacial;
- regras;
- banco de dados;
- transformações;
- execução conhecida.

5.2. IA

Útil para:

- linguagem natural;
- ambiguidades;
- classificação difícil;
- interpretação contextual;
- revisão;
- geração de explicações;
- interpretação de exceções;
- raciocínio sobre alternativas;
- elaboração de relatórios;
- assistência ao engenheiro.

6. Papel do Codex

Outro ponto que precisa ficar claro:

Codex não é o motor de IA em runtime do Auto AI Builder.

Codex foi utilizado como ferramenta de desenvolvimento.

Ele serve para:

- criar código;
- modificar código;
- escrever testes;
- depurar;
- refatorar;
- estruturar módulos;
- documentar;
- implementar funcionalidades.

Depois que o executável está rodando no computador do usuário, o Codex não está automaticamente “dentro dele”.

Se o produto precisar raciocinar usando IA durante a execução, será necessário utilizar:

- API;
 - modelo local;
 - outro serviço de IA;
 - ou interação assistida com ChatGPT.
-

7. Preferência atual sobre IA integrada

A preferência inicial do projeto foi evitar dependência forte de API.

Foi discutida a possibilidade de criar uma lateral semelhante a extensões do Google Sheets que permitisse trabalhar com ChatGPT.

Uma ideia considerada foi:

- botão para abrir ChatGPT Desktop;
- gerar automaticamente contexto;
- gerar prompt;
- copiar informações;
- utilizar Companion Window;
- retornar informações manualmente.

Isso evita inicialmente:

- cobrança de API;
- gerenciamento de chaves;
- infraestrutura de backend;
- complexidade adicional.

Entretanto:

integração bidirecional automática confiável com o aplicativo ChatGPT instalado não deve ser assumida como uma API disponível.

Por isso, para uma integração real de IA dentro do software, API ou modelo local continuam sendo caminhos possíveis.

8. Stack técnica proposta

A arquitetura discutida até agora considera:

- **SO:** Windows
- **Linguagem principal:** C#
- **Framework:** .NET 8
- **Desktop UI:** WPF
- **Persistência:** SQLite
- **AutoCAD:** plugin .NET / AutoLISP quando conveniente
- **Builder:** integração incremental, atualmente incluindo possibilidade de MCP oficial

Pasta usada durante desenvolvimento:

```
D:\Projetos\AutoAIBuilder
```

Estrutura aproximada histórica:

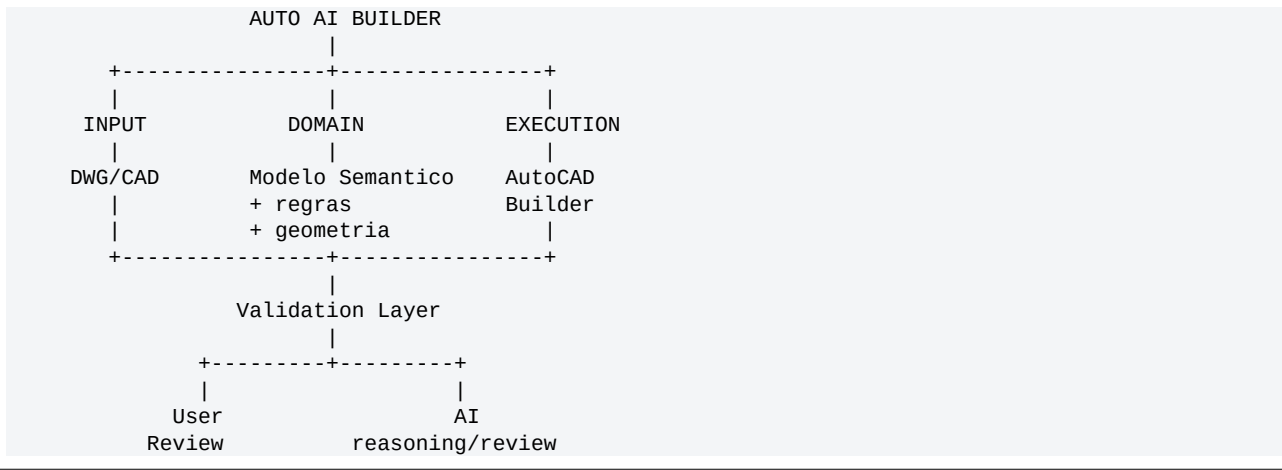
```
AutoAIBuilder/
|
+-- src/
+-- tests/
+-- docs/
|   +-- design/
+-- samples/
|   +-- autocad/
|   +-- builder/
+-- artifacts/
+-- tools/
```

Para colaboração futura, a recomendação é expandir para algo próximo de:

```
AutoAIBuilder/
|
+-- README.md
+-- PROJECT_HANDOFF.md
+-- CONTEXT.md
+-- ARCHITECTURE.md
+-- DECISIONS.md
+-- ROADMAP.md
+-- CHANGELOG.md
|
+-- docs/
|   +-- autocad/
|   +-- builder/
|   +-- mcp/
|   +-- computer-use/
|   +-- algorithms/
|   +-- tests/
|
+-- src/
+-- tests/
+-- prototypes/
+-- samples/
+-- artifacts/
+-- tools/
```

9. Arquitetura lógica pretendida

Uma forma simplificada de visualizar:



10. Modelo intermediário

Uma das decisões mais importantes foi não permitir que AutoCAD e Builder fossem diretamente dependentes um do outro.

Entre eles deverá existir um **modelo intermediário do projeto**.

Esse modelo deve representar semanticamente o projeto.

Exemplo conceitual:

```

Project
+-- Floors
|   +-- Rooms
|   +-- Walls
|   +-- Openings
|   +-- Points
|
+-- ElectricalPoints
+-- HydraulicPoints
+-- SanitaryPoints
+-- TextAnnotations
+-- Associations
+-- Circuits
+-- Conduits
+-- ValidationIssues
  
```

Um ponto poderá futuramente conter algo como:

```

Point
+-- ID
+-- CAD Handle
+-- Discipline
+-- Type
+-- Position X/Y/Z
+-- Rotation
+-- Height
+-- Voltage
+-- Current
+-- Power
+-- Layer
+-- Room
+-- Associated Texts
+-- Confidence
+-- Source
+-- Validation Status
+-- Builder Mapping
  
```

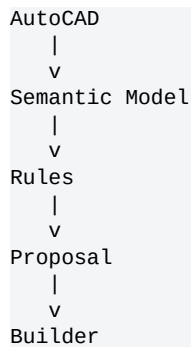
10.1. Por que o modelo intermediário é importante

Sem ele, o projeto vira:

“Clique aqui no AutoCAD e depois clique ali no Builder.”

Isso não escala.

Com o modelo intermediário:



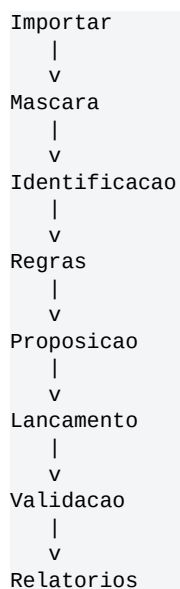
O AutoCAD passa a ser uma fonte de dados. O Builder passa a ser um destino/executor. A inteligência fica no Auto AI Builder.

11. Interface pretendida

Foi discutido e aprovado conceitualmente um layout desktop com:

- tema escuro;
- navegação lateral;
- fluxo/etapas visíveis;
- planta central;
- painel de agentes/tarefas;
- cards inferiores;
- painel de revisão;
- indicadores de confiança.

Fluxo principal:



11.1. Painel de revisão

O usuário deverá conseguir selecionar um elemento e visualizar informações como:

Tomada #T042

Tipo: Tomada dupla
 Altura: Media
 Corrente: 20 A
 Tensao: 220 V
 Orientacao: 219.31 graus
 Ambiente: Area de servico

DWG Handle: 2AFD76

Confianca:
 Tipo: 99%
 Tensao: 97%
 Orientacao: 100%

Status:
 - Identificada
 - Associada
 - Ainda nao lancada

Possíveis ações:

[Aprovar]
 [Editar]
 [Ignorar]
 [Ver no desenho]

12. Conceito de agentes

Uma arquitetura por agentes foi discutida como evolução futura.

12.1. CAD Agent

Responsável por:

- leitura do desenho;
- organização;
- layers;
- blocos;
- entidades;
- coordenadas.

12.2. Electrical Agent

Responsável por:

- pontos elétricos;
- tomadas;
- iluminação;
- circuitos;
- cargas.

12.3. Hydraulic Agent

Responsável por:

- pontos de água;
- equipamentos;
- conexões;
- redes.

12.4. Sanitary Agent

Responsável por:

- pontos de esgoto;
- ralos;
- tubulações;
- aparelhos.

12.5. Reviewer Agent

Responsável por:

- inconsistências;
- conflitos;
- elementos sem associação;
- baixa confiança;
- revisão de decisões.

12.6. Orchestrator

Responsável por:

- controlar a sequência;
- distribuir tarefas;
- gerenciar estado;
- exigir aprovação.

Esses “agentes” não precisam necessariamente ser LLMs independentes. Muitos podem simplesmente ser **módulos especializados do software**.

13. Primeiro grande experimento - preparação de máscara

Esse foi o primeiro problema atacado em profundidade.

Objetivo:

Receber um DWG arquitetônico complexo e identificar quais entidades são úteis para o projeto complementar.

O desenho analisado possuía aproximadamente **15.400 objetos** segundo os registros iniciais.

Durante uma etapa de triagem, houve uma classificação histórica de aproximadamente:

MANTER	= 900
REMOVER	= 448
REVISAR	= 52

Esses números pertencem a uma fase preliminar e não devem ser tratados como a contagem final do modelo semântico.

14. Classificação V0.3.1

Outra fotografia histórica do classificador produziu aproximadamente:

ELE	311
HID	119

SAN	2
ARQ_BASE	2
MOBILIARIO	118
CONTEXTO	354
DESCARTAVEIS	447
REVISAR	47

Total do conjunto categorizado:

1.400 entidades

Esses números representam um estágio anterior ao refinamento das regras.

15. Problemas identificados na fase inicial de classificação

O algoritmo precisava entender que nem toda entidade desenhada deveria virar ponto de projeto.

Exemplos encontrados:

- elementos de legenda;
- vistas A-D;
- indicadores gráficos;
- logotipos;
- nomes de desenho;
- mobiliário;
- detalhes.

Também houve confusão envolvendo:

- símbolos reais;
- símbolos da legenda;
- textos 20A;
- textos 220V;
- tomadas duplas;
- blocos arquitetônicos.

16. Regra importante - arquitetura não deve ser destruída

Um erro conceitual inicial seria simplesmente “explodir tudo”. Isso foi rejeitado.

O sistema deve preservar:

- blocos;
- estrutura do desenho;
- atributos;
- layers;
- relações;
- cores úteis.

Especialmente porque muita informação semântica pode estar justamente nesses elementos.

17. CINZA PONTOS

Um elemento importante do desenho foi identificado como:

CINZA PONTOS

Regra definida:

CINZA PONTOS deve ser tratado como base/contexto arquitetônico.

Não deve ser interpretado internamente como se cada entidade dentro dele representasse pontos de instalações.

Também não deve ser alterado destrutivamente.

18. Outros elementos de base/contexto

Entre elementos tratados como contexto estavam:

CINZA PONTOS
BASE 1
RFWERF

O bloco RFWERF foi interpretado como mobiliário/contexto.

19. Vistas e elementos descartáveis

Foi decidido que elementos como:

- vistas A;
- vistas B;
- vistas C;
- vistas D;
- marcadores;
- detalhes genéricos;
- logo;
- nome do desenho;

poderiam ser descartados do fluxo semântico.

20. Sanitário no arquivo de teste

Em uma versão do classificador apareceram:

SAN = 2

Posteriormente verificou-se que esses dois elementos eram detalhes genéricos e **não pontos sanitários reais do projeto**.

Conclusão:

Naquele arquivo específico, não existiam pontos sanitários relevantes identificados naquele estágio.

21. Resultado V0.4 - marco importante

Depois das correções, chegou-se a uma fotografia muito mais confiável.

Foram identificados:

272 pontos principais

divididos em:

Eletricos	196
Hidraulicos	76

Total	272

Esse conjunto passou a ser utilizado como referência para regressão.

22. Componentes associados

Além dos 272 pontos principais, existiam elementos auxiliares associados.

Exemplos:

- triângulos gráficos;
- textos;
- indicações de corrente;
- tensão;
- partes de tomadas duplas.

O snapshot consolidado posterior chegou a:

272 pontos principais
+
100 componentes / textos associados
=
372 objetos semanticamente relevantes

Este é um dos conjuntos de referência mais importantes do projeto.

23. V0.4 - dados históricos adicionais

Em determinados snapshots da V0.4 foram registrados componentes como:

ELE_GRAF
ELE_TEXT0
HID_GRAF

Uma fotografia consolidada posteriormente considerava aproximadamente:

ELE_GRAF	24
ELE_TEXT0	70
HID_GRAF	6

Total	100

Durante o desenvolvimento existiram contagens intermediárias ligeiramente diferentes - especialmente em tomadas duplas e textos.

Por isso:

Não usar automaticamente todos os números históricos como assertions simultâneos.

O dataset canônico deve ser formalizado no repositório.

24. Tomadas duplas - bug importante

Um problema significativo apareceu na identificação de tomadas duplas.

A tomada dupla possuía mais de um componente gráfico.

Inicialmente:

apenas parte da tomada estava sendo considerada.

Correção conceitual:

uma tomada dupla precisa carregar os dois elementos gráficos associados.

Além disso, os textos associados também devem acompanhá-la.

Exemplo:

```
[triangulo 1]
[triangulo 2]
[20A]
[220V]
```

Não basta identificar somente um dos triângulos.

25. Textos 20 A e 220 V

Outro aprendizado importante foi que textos como:

```
20A
220V
```

não podem ser tratados como texto descartável.

Eles alteram semanticamente o ponto.

O sistema precisa associar:

```
Point
+-- Current = 20 A
+-- Voltage = 220 V
```

e não simplesmente preservar o texto visual.

26. Problema com arquitetura e textos

Em determinado estágio, alguns textos 20A e 220V acabaram sendo absorvidos ou tratados como parte da arquitetura.

Isso evidenciou uma regra:

contexto arquitetônico não pode simplesmente engolir textos técnicos espacialmente relacionados aos pontos.

A associação precisa ser semântica e espacial.

27. Baixa confiança

O sistema passou a separar entidades de baixa confiança.

Em uma fase foram excluídos aproximadamente:

59 objetos de baixa confiança

A filosofia definida foi:

Em caso de dúvida, enviar para revisão em vez de inventar uma classificação.

28. V0.5 - máscara semântica

A V0.5 representou uma evolução importante.

Em vez de somente selecionar entidades, o programa passou a organizar elementos em **layers semânticas**.

Resultado registrado:

372 objetos temporariamente selecionados
272 pontos
100 componentes/textos
35 layers semânticas
0 erros de layer

E:

0 entidades apagadas
0 blocos explodidos

Isso está alinhado com a filosofia não destrutiva.

29. Exemplos de layers semânticas

Um exemplo de nomenclatura discutida:

PONTOS_ELE_TOMADA_DUPLA_20A_220V

Também existiam categorias relacionadas a:

- tomada;
- tomada dupla;
- cabeceira;
- teto;
- rede/dados;
- chuveiro;
- ducha;
- lava-louças;
- ralo;
- torneira.

A ideia central:

o DWG deixa de ser somente uma coleção de desenhos e passa a carregar uma organização semântica compreensível pelo software.

30. Por que criar layers semânticas

O Builder não preservava necessariamente a organização original da máscara da forma desejada.

Uma solução encontrada foi criar layers específicas para:

- tomadas;
- interruptores;
- luminárias;
- hidráulicos;
- etc.

Assim o arquivo importado no Builder passa a carregar uma informação muito mais útil.

31. V0.6 - limpeza

A limpeza da máscara evoluiu posteriormente.

Regra:

operações destrutivas somente em arquivo explicitamente identificado como cópia/máscara.

Exemplos esperados no nome:

MASCARA
COPIA

Antes de remover:

- validar layers;
- apresentar seleção;
- solicitar confirmação.

Comando conceitual:

REMOVED

após confirmação explícita.

32. Problema ainda existente: cotas

Uma das rotinas de limpeza não removeu todas as cotas.

Isso foi conhecido e deliberadamente não priorizado naquele momento.

Conclusão:

Não considerar limpeza de cotas como problema resolvido.

Pode haver necessidade de correção manual ou nova regra.

33. Exportação estruturada

A máscara não deveria ser o único resultado.

Foi proposto exportar também informações estruturadas.

Exemplo:

CSV / JSON

contendo:

ID
Handle
Tipo
X
Y
Z
Rotacao
Layer
Ambiente
Altura
Corrente
Tensao
Confianca
Associacoes

Isso permitiria que Auto AI Builder, AutoCAD e Builder compartilhassem um modelo comum.

34. Tarefa 11.6C

Uma das tarefas de desenvolvimento mais importantes registradas foi chamada:

11.6C

Ela estava relacionada à consolidação do modelo semântico.

Entre as necessidades estavam:

- importar CSVs históricos;
- estruturar modelos de pontos;
- estruturar componentes;
- estruturar direções;
- validar contagens;
- validar handles;
- validar unidades;
- validar associações;
- migrar schema do SQLite;
- criar revisão visual;
- testes de regressão.

Dataset esperado para regressão:

272 pontos
100 componentes

A recomendação de esforço na época foi utilizar raciocínio mais profundo para implementar essa parte devido ao impacto estrutural.

35. Testes de regressão

Os 272 + 100 objetos são importantes porque representam um primeiro **ground truth** do sistema.

Uma mudança futura no classificador não pode simplesmente produzir algo diferente sem explicar o porquê.

Exemplo:

```
Expected:
Points    = 272
Components = 100
Total     = 372
```

Idealmente o repositório deverá possuir:

```
tests/regression/mask_v05/
```

com:

- DWG de teste;
 - JSON esperado;
 - handles esperados;
 - classificação;
 - associações;
 - métricas.
-

36. Handles conhecidos

Handles do AutoCAD foram considerados importantes como identificadores estáveis dentro do DWG.

Alguns casos que entraram em revisão:

```
22FAD6F
21AE333
```

Em determinado teste apresentavam ângulo próximo de:

```
222.234 graus
```

Esses handles foram separados para revisão.

37. Segundo grande experimento - lançamento no Builder

Depois da máscara, o projeto avançou para:

transformar pontos identificados no AutoCAD em lançamentos dentro do Builder.

Esse é um problema diferente.

Não basta saber que existe uma tomada.

É necessário saber:

- qual peça utilizar;
 - qual rede;
 - qual altura;
 - onde clicar;
 - qual referência usar;
 - qual orientação;
 - qual offset aplicar.
-

38. Fluxo manual observado no Builder

O lançamento manual seguia aproximadamente:

1. abrir projeto de Fiação;
 2. selecionar pavimento;
 3. acessar Lançamento;
 4. confirmar rede Elétrica;
 5. definir Posição;
 6. escolher a peça;
 7. utilizar Ponto relativo;
 8. clicar na referência;
 9. definir deslocamento;
 10. indicar orientação;
 11. finalizar;
 12. validar visualmente.
-

39. Erro operacional recorrente

Durante os testes, um erro apareceu mais de uma vez:

esquecer de configurar Posição = Média.

Conclusão:

Qualquer executor automático deve verificar explicitamente:

```
Network
Position
Part
Command
```

antes de lançar.

Nunca presumir estado anterior da interface.

40. Peça utilizada nos testes

Para uma tomada média 20 A foi utilizada:

```
Pontos de força
-> Uso geral
-> 2P+T 20 A
-> media
```

41. Regra 10 A / 20 A

Um cuidado importante:

Não escolher automaticamente uma peça apenas pelo valor de potência apresentado pelo Builder.

A especificação deve vir do projeto/máscara.

Exemplo:

20 A

é uma propriedade do ponto.

Não deve ser deduzida exclusivamente de uma carga default como 2200 W.

42. Problema geométrico descoberto

O ponto visual do símbolo no AutoCAD não coincide necessariamente com o ponto de inserção necessário no Builder.

Isso gerou uma descoberta importante:

existem pelo menos duas transformações geométricas independentes.

Transformação A

Sistema de coordenadas DWG
|
v
Sistema de coordenadas Builder

Transformação B

Centro / referencia do simbolo
|
v
Ponto real de insercao

A transformação B depende de:

- orientação;
- símbolo;
- altura;
- tipo de peça.

Elas não devem ser misturadas.

43. Offsets locais identificados

Para tomadas médias, foram testados offsets aproximadamente:

1,5 cm
2,4 cm

Um caso validado para orientação à esquerda utilizou:

2.4, -1.5, 0

44. Mapeamento de orientação testado/inferido

Uma tabela histórica foi construída:

Baixo	->	+1.5	, +2.4
Esquerda	->	+2.4	, -1.5
Cima	->	-1.5	, -2.4
Direita	->	-2.4	, +1.5

Importante:

- os casos **baixo** e **esquerda** receberam validação prática;
 - os casos **cima** e **direita** foram inicialmente derivados por inversão geométrica e precisam ser tratados como **a validar experimentalmente**.
-

45. Primeira tomada validada

Um caso de tomada orientada para baixo foi calibrado.

Handle registrado:

21AF02F

Rotação:

90.02296445 graus

Após ajustes locais, o lançamento ficou corretamente posicionado.

46. Tomada inclinada sudoeste

Outro teste particularmente importante foi uma tomada inclinada para sudoeste.

Handle:

2AFD76

Rotação:

219.31025869 graus

A orientação inclinada não deve ser convertida obrigatoriamente para uma direção cardinal.

Regra:

preservar o ângulo real quando necessário.

47. Transformação global observada

Durante esse segundo lançamento, inicialmente foi necessário corrigir aproximadamente:

23,7 cm para a esquerda
25,2 cm para baixo

Depois houve uma correção residual local de aproximadamente:

1,5 cm para cima
2,4 cm para a esquerda

Ao final o resultado foi considerado:

“Lançamento perfeito.”

Essa experiência demonstrou empiricamente a existência das duas transformações descritas anteriormente.

48. Cuidado com os números de offset

Os valores:

23,7 cm
25,2 cm

não devem ser hardcoded como “offset padrão”.

Eles eram parte de um alinhamento daquele caso.

Precisamos obter a transformação global de forma matemática.

Já:

1,5 cm
2,4 cm

parecem representar deslocamento local relacionado ao ponto de inserção da peça e merecem virar parâmetros calibráveis por família/orientação.

49. Piloto de tomadas

Em uma análise posterior foram identificadas aproximadamente:

38 tomadas medias

distribuídas em:

Direita	12
Cima	11
Esquerda	9
Baixo	6

Além disso:

24 cardinais
14 inclinadas

Esse dataset pode ser excelente para calibração futura.

50. Direções

As orientações cardinais podem ser tratadas aproximadamente como:

0 graus
90 graus
180 graus
270 graus

Mas o software não deve forçar todos os símbolos para esses ângulos.

Exemplo real:

219.310 graus

deve permanecer inclinado.

51. Situação da automação geral de lançamento

Apesar dos casos bem-sucedidos:

não existe ainda uma transformação universal AutoCAD -> Builder completamente validada.

Existem:

- provas de conceito;
- calibrações;
- offsets conhecidos;
- casos validados.

Ainda falta formalizar matematicamente o processo.

52. Status das tomadas duplas no lançamento

A identificação semântica evoluiu, mas o fluxo completo de lançamento de:

- tomadas duplas;
- textos;
- todas as variações;

não deve ser considerado integralmente resolvido.

Esse é um item de regressão importante.

53. Builder - decisão de pausa

Em determinado momento, Danilo iniciou um projeto real.

Foi decidido:

pausar temporariamente a automação interna no Builder.

Durante esse período:

- criação da edificação;
- configuração;
- importação da máscara;

continuaram sendo feitas manualmente.

Isso não significa que a ideia foi abandonada.

Foi uma decisão operacional para não transformar um projeto de cliente em laboratório.

54. Comunicação AutoCAD -> Builder

Também foi decidido que a comunicação completamente automática AutoCAD -> Builder não deveria ser prioridade imediata enquanto o modelo semântico ainda estivesse sendo consolidado.

Prioridade:

Entender corretamente
antes de
Executar automaticamente

55. Computer Use

Computer Use foi utilizado como ferramenta exploratória para entender se um agente poderia operar interfaces gráficas.

Isso ajudou a:

- entender fluxo do Builder;
- identificar estados;
- executar testes;
- documentar sequências.

Entretanto, Computer Use possui limitações inerentes:

- depende de interface;
 - depende de janela ativa;
 - UI pode mudar;
 - coordenadas podem mudar;
 - modal pode aparecer;
 - latência;
 - seleção incorreta;
 - risco de clique errado.
-

56. Estratégia defensiva para automação de GUI

Caso UI Automation ou automação visual ainda sejam necessárias, o executor precisa possuir:

```
Check state
-> Act
-> Verify
-> Log
-> Continue
```

E nunca:

```
Click
Click
Click
Click
Click
```

sem validação.

57. Recursos obrigatórios de segurança

Qualquer automação do Builder deveria futuramente incluir:

- botão de abortar;
- checkpoint;
- log;
- captura de erro;

- validação de estado;
 - timeout;
 - recuperação;
 - confirmação visual;
 - modo passo-a-passo;
 - velocidade reduzida durante testes.
-

58. Perfis de versão do Builder

Caso seja necessário controlar a interface visual do Builder, provavelmente será necessário ter alguma forma de:

```
BuilderProfile
+-- version
+-- menu structure
+-- selectors
+-- shortcuts
+-- expected states
```

porque atualizações de software podem alterar a UI.

59. Mudança importante em setembro de 2026 - MCP do Builder

Esta é provavelmente a maior mudança recente em relação às premissas iniciais do projeto.

No começo, trabalhávamos praticamente sob a hipótese de que o Builder não oferecia uma interface programática adequada para o que queríamos fazer.

No final de setembro de 2026 foi configurado um **MCP oficial do AltoQi Builder**.

Endpoint utilizado:

```
https://mcp.altoqi.com.br/builder/mcp
```

OAuth Client ID:

```
altoqi-connect
```

Também apareceu no fluxo de configuração:

```
User-Defined OAuth Client
```

e configuração com:

```
client_secret_post
```

callback:

```
https://chatgpt.com/connector_platform_oauth_redirect
```

60. Requisitos observados do MCP

Para a integração funcionar foi observado que é necessário:

- Builder em execução;
- projeto aberto;
- sessão/licença autenticada;
- AltoQi Axis / MCP conectado;

- reconectar quando necessário após reiniciar o Builder.

61. Experimento com circuitos via MCP

Em 29/09/2026 foi tentado consultar:

os circuitos elétricos do projeto.

A chamada não conseguiu retornar os dados porque não havia uma sessão ativa do Builder devidamente vinculada ao MCP naquele momento.

Isso é importante.

O erro **não demonstrou que o MCP não consegue acessar circuitos.**

Demonstrou que:

a integração depende de uma sessão Builder ativa.

62. Consequência arquitetural do MCP

Pablo deve considerar o MCP como uma prioridade de investigação.

Ele pode mudar significativamente a arquitetura anterior.

Antes:

```
Auto AI Builder
  |
  v
UI Automation
  |
  v
Builder
```

Possibilidade atual:

```
Auto AI Builder
  |
  v
MCP / integracao oficial
  |
  v
Builder
```

OU:

```
AutoAIBuilder --- MCP
AutoAIBuilder -|
AutoAIBuilder --- UI Automation fallback
```

Se o MCP oferecer criação e modificação suficientemente profundas, podemos eliminar grande parte da fragilidade relacionada a mouse/teclado/visão.

63. Primeira tarefa técnica recomendada para Pablo

Antes de desenvolver mais automação visual do Builder:

mapear completamente a superfície do MCP AltoQi Builder.

Perguntas que precisam ser respondidas:

- quais tools estão disponíveis?
- quais dados podem ser lidos?
- quais dados podem ser escritos?
- consegue listar pavimentos?
- consegue listar circuitos?
- consegue criar circuito?
- consegue criar ponto?
- consegue editar ponto?
- consegue lançar conduto?
- consegue obter coordenadas?
- consegue identificar peças?
- consegue manipular layers?
- consegue consultar rede hidráulica?
- consegue executar cálculo?
- consegue exportar informações?
- existe schema estável?
- como é tratada versão?
- como funciona autenticação?
- como funciona sessão?

Esse trabalho pode economizar meses de automação de interface.

64. Não assumir que MCP resolve tudo

Também não devemos partir do extremo oposto.

Ainda não sabemos se o MCP:

- cobre todas as funcionalidades;
- permite escrita;
- é estável;
- é oficialmente suportado para automações extensas;
- expõe geometria suficiente;
- funciona em todos os módulos.

Portanto:

investigar antes de reescrever arquitetura.

65. AutoCAD

Diferentemente do Builder, AutoCAD oferece caminhos programáticos maduros.

Preferência:

C#/.NET Plugin

E, quando adequado:

AutoLISP

A automação do AutoCAD **não deve depender primariamente de visão computacional.**

66. Responsabilidades do plugin AutoCAD

O plugin poderá realizar:

- leitura de entities;
- blocks;
- layers;
- handles;
- texts;
- rotations;
- geometry;
- attributes;
- coordinates;
- selection;
- criação de layers;
- modificação controlada;
- exportação.

Idealmente a interface AutoCAD -> Auto AI Builder será estruturada.

67. Banco de dados

SQLite foi escolhido conceitualmente porque atende bem ao caso:

- desktop;
- local;
- sem servidor;
- fácil backup;
- transacional;
- suficientemente robusto.

Pode armazenar:

```
Projects
Drawings
Entities
Points
Associations
Rules
Transformations
ExecutionLogs
Snapshots
ValidationIssues
```

68. Versionamento do modelo

Qualquer modelo intermediário persistido deve carregar versão.

Exemplo:

```
schemaVersion: 3
```

Isso será importante porque:

- classificação evolui;
- regras evoluem;

- campos evoluem;
- transforms evoluem.

Não depender de migração manual improvisada.

69. Snapshots

Antes de operações significativas:

```
Snapshot A
|
v
Automation
|
v
Snapshot B
```

Isso facilita:

- regressão;
 - comparação;
 - rollback;
 - auditoria;
 - debugging.
-

70. Logs

Um lançamento deveria registrar algo conceitualmente semelhante a:

```
2026-10-01 14:32:13

Action:
CreateElectricalOutlet

Source:
CAD Handle 2AFD76

Classification:
20A / Medium / 220V

Position:
X = ...
Y = ...

Rotation:
219.310 graus

Builder:
Piece = ...

Result:
SUCCESS

Validation:
VISUAL_CONFIRMED
```

71. Confiança

Classificações não determinísticas deveriam possuir confiança.

Exemplo:

```
Outlet classification: 0.99
Voltage association: 0.97
Room association: 0.88
```

Uma política futura poderia ser:

```
> 0.98      automatic
0.85-0.98   quick review
< 0.85      manual review
```

Os thresholds exatos ainda não foram definidos e devem ser configuráveis.

72. Regras vs aprendizado

O produto não deve “aprender sozinho” silenciosamente com qualquer correção.

Correções humanas podem futuramente alimentar aprendizado, mas por meio de um processo controlado:

```
User correction
  |
  v
Candidate rule/update
  |
  v
Validation
  |
  v
Regression suite
  |
  v
Release
```

73. Condutos

Lançamento automático de condutos está na visão do produto, porém ainda não foi validado no mesmo nível das tomadas.

Provavelmente exigirá:

- localização dos pontos;
- arquitetura;
- paredes;
- proximidade;
- quadro;
- circuitos;
- caminhos;
- restrições;
- regras;
- geometria;
- otimização.

73.1. Princípio para condutos

Não começar tentando produzir a rota perfeita.

Uma estratégia mais realista:

```

AI/rules propose
  |
  v
Engineer reviews
  |
  v
System executes

```

Posteriormente aumentar autonomia.

74. Circuitos

Circuitos fazem parte da visão futura.

O sistema deverá futuramente trabalhar com:

- quadro;
- cargas;
- pontos;
- tensão;
- fases;
- agrupamento;
- critérios de circuito;
- distância;
- equilíbrio;
- regras de projeto.

O MCP pode transformar significativamente essa etapa.

75. Hidráulico

O pipeline deverá eventualmente funcionar também para hidráulica.

Já existiram classificações de pontos como:

- torneira;
- chuveiro;
- ducha;
- lava-louças.

No dataset principal:

```
76 pontos hidraulicos
```

foram tratados como pontos válidos.

76. Sanitário

A disciplina sanitária está planejada, mas os dois itens inicialmente classificados como sanitários no arquivo de teste não eram pontos reais.

Portanto não existe ainda dataset equivalente ao elétrico/hidráulico para validar a disciplina sanitária.

77. Estrutural / TQS

O Auto AI Builder tem potencial de evoluir para estrutural/TQS porque esse também faz parte do workflow profissional.

Porém:

não deve ser prioridade antes do núcleo AutoCAD + Builder estar maduro.

78. Arquitetura do MVP

Um MVP razoável não precisa projetar automaticamente um edifício inteiro.

Uma definição muito mais útil:

1. Importar DWG
2. Identificar pontos
3. Exibir overlay/revisão
4. Corrigir classificação
5. Exportar modelo semântico
6. Conectar ao Builder
7. Lançar um tipo de ponto
8. Validar resultado
9. Registrar auditoria

Se isso funcionar de ponta a ponta de forma consistente, temos um produto real.

79. O que já pode ser considerado validado

79.1. Máscara

[VALIDADO] Leitura e classificação de entidades é viável.

[VALIDADO] Separação entre pontos principais e componentes é viável.

[VALIDADO] Foram obtidos 272 pontos principais de referência.

[VALIDADO] Foram obtidos 100 componentes associados em snapshot consolidado.

[VALIDADO] Organização em layers semânticas é viável.

[VALIDADO] 35 layers semânticas foram criadas em uma versão.

[VALIDADO] Processo pode ser não destrutivo.

79.2. Associações

[VALIDADO] Textos próximos podem ser relacionados semanticamente aos pontos.

[VALIDADO] 20A e 220V são informações relevantes.

[VALIDADO] Tomadas duplas precisam carregar múltiplos componentes.

79.3. Lançamento

[VALIDADO] É possível lançar pontos no Builder seguindo fluxo automatizável.

[VALIDADO] Offset entre símbolo e ponto real de inserção existe.

[VALIDADO] Caso de tomada orientada para baixo funcionou.

[VALIDADO] Caso inclinado sudoeste funcionou.

[VALIDADO] Correção visual chegou a um lançamento considerado perfeito.

79.4. Produto

[VALIDADO] Arquitetura híbrida é tecnicamente viável.

[VALIDADO] Núcleo determinístico pode executar grande parte do trabalho.

[VALIDADO] IA não precisa controlar tudo.

80. Em desenvolvimento / não validado totalmente

[EM DESENVOLVIMENTO] Transformação universal DWG -> Builder.

[EM DESENVOLVIMENTO] Offsets para todas as orientações.

[EM DESENVOLVIMENTO] Biblioteca completa de peças.

[EM DESENVOLVIMENTO] Lançamento completo de tomadas duplas.

[EM DESENVOLVIMENTO] Associação universal 20A/220V.

[EM DESENVOLVIMENTO] Overlay gráfico.

[EM DESENVOLVIMENTO] Modelo intermediário definitivo.

[EM DESENVOLVIMENTO] Schema SQLite definitivo.

[EM DESENVOLVIMENTO] Plugin AutoCAD definitivo.

[EM DESENVOLVIMENTO] Integração MCP Builder.

[EM DESENVOLVIMENTO] Executor Builder robusto.

[EM DESENVOLVIMENTO] Condutos.

[EM DESENVOLVIMENTO] Circuitos.

[EM DESENVOLVIMENTO] Hidráulica automatizada.

[EM DESENVOLVIMENTO] Sanitário.

[EM DESENVOLVIMENTO] Agentes inteligentes.

[EM DESENVOLVIMENTO] Chat lateral.

[EM DESENVOLVIMENTO] Instalador.

[EM DESENVOLVIMENTO] Licenciamento.

81. Abordagens que não devem ser repetidas

81.1. Explodir indiscriminadamente o DWG

[NAO REPETIR] Não fazer. Isso destrói informação útil.

81.2. Cinzar tudo / perder estrutura

[NAO REPETIR] Não fazer. Layers, blocos e cores carregam informação.

81.3. Ler CINZA PONTOS como instalações reais

[NAO REPETIR] Não fazer. É contexto arquitetônico.

81.4. Tratar textos técnicos como lixo

[NAO REPETIR] Não fazer. 20A e 220V alteram significado do ponto.

81.5. Usar somente Computer Use para AutoCAD

[NAO REPETIR] Evitar. AutoCAD possui APIs muito melhores.

81.6. Fazer Codex virar “IA interna” do aplicativo

[NAO REPETIR] Conceitualmente incorreto. Codex desenvolve o programa; não é automaticamente runtime.

81.7. Automação Builder cega

[NAO REPETIR] Não usar sequência de cliques sem confirmação.

81.8. Misturar offset global com offset da peça

[NAO REPETIR] São problemas matemáticos distintos.

82. Questões em aberto

82.1. DWG

- Qual parser/abstração será canônico?
- Plugin AutoCAD ou leitura externa?
- Qual formato do semantic model?
- Como versionar?
- Como tratar XREF?
- Como tratar blocos dinâmicos?

82.2. Associação

- Qual algoritmo liga texto e ponto?
- nearest neighbor?
- cone angular?
- bounding box?
- score ponderado?

82.3. Transformação

- Como obter a transformação DWG -> Builder?
- affine transform?
- translation?
- rotation?
- scale?
- anchors?

82.4. Builder

- O MCP permite write?
- Quais entidades são acessíveis?
- Como identificar o mesmo ponto após criação?
- Existe ID?
- Existe coordenada absoluta?

82.5. IA

- API é realmente necessária?
- modelo local?
- nenhum modelo no MVP?

82.6. UX

- como apresentar 500 pontos sem sobrecarregar?
 - como revisar em lote?
-

83. Hipótese matemática para transformação

Esse é um ponto que vale Pablo estudar.

A transformação não deveria ser calibrada “na mão” para cada ponto.

Idealmente selecionar pontos comuns:

```
DWG:
P1(x1,y1)
P2(x2,y2)
P3(x3,y3)

Builder:
P1'(x1',y1')
P2'(x2',y2')
P3'(x3',y3')
```

e estimar:

```
rotation
translation
scale
```

Se os sistemas forem equivalentes, uma transformação rígida/afim simples poderá resolver a parte global.

Depois aplicar offset local da peça.

84. Estrutura ideal do pipeline geométrico

```
CAD coordinates
|
v
Global Transform
|
v
Builder reference coordinates
|
v
Piece Local Offset
|
v
Final insertion point
```

Essa separação é crítica.

85. Overlay

Antes de lançar no Builder, seria extremamente útil visualizar:

```
CAD
+
Detected semantic objects
+
labels
+
confidence
```

Exemplo conceitual:

```
[verde] confirmado
[amarelo] revisar
[vermelho] conflito
```

Esse visualizador reduz drasticamente o risco de erro.

86. Estratégia recomendada de desenvolvimento

Evitar construir várias disciplinas simultaneamente.

Sugestão:

```
Vertical Slice #1

DWG
|
v
Tomada
|
v
Semantic Model
|
v
Review
|
v
Builder
|
v
Validation
```

Depois expandir.

87. Ordem recomendada atualmente

Considerando todo o aprendizado existente e principalmente a descoberta do MCP:

Etapa A - congelar baseline

Formalizar:

```
272 points
100 components
```

em fixtures/testes.

Etapa B - auditar código atual

Descobrir:

- qual versão realmente está no repo;
- o que está implementado;
- o que é protótipo;
- o que é script isolado;
- o que nunca foi integrado.

Etapa C - modelo de domínio

Formalizar classes/interfaces.

Etapa D - importador DWG

Produzir semantic model reproduzível.

Etapa E - regression suite

Garantir que mudanças não quebrem V0.5.

Etapa F - visualizador

Permitir review.

Etapa G - MCP spike

Mapear Builder MCP.

Etapa H - integração mínima Builder

Exemplo:

```
Read current project
-> list points
-> create one point
-> verify
```

caso permitido.

Etapa I - lançar tomadas

Automatizar uma família por completo.

Etapa J - expandir

Somente depois:

- interruptores;
 - luminárias;
 - circuitos;
 - condutos;
 - hidráulico.
-

88. Definition of Done para tomada automática

Um lançamento de tomada só deveria ser considerado resolvido quando:

```

Input DWG
|
v
Point detected
|
v
Attributes detected
|
v
User can review
|
v
Builder receives correct piece
|
v
Correct coordinate
|
v
Correct height
|
v
Correct rotation
|
v
Correct electrical attributes
|
v
System verifies
|
v
Audit recorded
  
```

e isso ocorre repetidamente, não apenas em uma tomada.

89. Métricas futuras

Sugestões de métricas importantes:

```

Point detection recall
Point detection precision

Text association accuracy

Correct type
Correct current
Correct voltage
Correct height
Correct orientation

Builder placement success

Manual corrections / 100 points

Time saved / project
  
```

90. Critério de sucesso do produto

O objetivo não precisa ser:

“100% do projeto sem intervenção humana.”

Um resultado extremamente valioso poderia ser:

90-95% das tarefas repetitivas automatizadas
+
engenheiro revisando decisoes criticas

Se um projeto que leva horas de preparação e lançamento passar a exigir minutos de revisão, o produto já terá enorme valor.

91. Visão do front-end

O usuário não deve precisar ser programador.

Workflow ideal:

```

Novo Projeto

[Importar arquitetura]

  |
  v

Analisando...

Pontos detectados: 272

Eletricos: 196
Hidraulicos: 76

Revisao necessaria: 8

[Revisar]
[Aprovar]
```

Depois:

```

Builder conectado

[Executar lancamentos]

Tomadas
██████████ 82%

163/196
```

No produto final, o indicador de progresso poderá ser representado por componentes visuais adequados, não necessariamente pelos caracteres acima.

92. Painel “Assistente IA”

Foi imaginada também uma aba lateral:

Assistente IA

semelhante à experiência de extensões com ChatGPT.

Possíveis funções:

- explicar erro;
- sugerir classificação;
- gerar regra;
- interpretar projeto;

- revisar circuito;
- responder perguntas;
- gerar relatório.

Inicialmente pode funcionar como uma integração assistida.

93. Segurança de API

Caso API seja adotada:

Nunca salvar chave diretamente em:

```
appsettings.json
```

ou no repositório.

Utilizar armazenamento seguro do Windows.

A assinatura ChatGPT Plus não equivale a créditos de API.

API possui cobrança independente.

94. Custos de IA

Foi discutida anteriormente uma arquitetura que use IA somente quando necessário.

Exemplo:

```
Local/deterministic:
90+% das operacoes
```

```
IA:
5-10% das situacoes ambigüas
```

Isso reduz:

- custo;
 - latência;
 - dependência externa;
 - risco.
-

95. Conectividade

O software idealmente deve conseguir continuar realizando tarefas básicas mesmo sem internet:

- leitura;
- máscara;
- classificação determinística;
- geometria;
- banco;
- revisão.

IA em nuvem seria uma capacidade adicional.

96. Licenciamento

Licenciamento e assinatura comercial foram discutidos, inclusive potencial de cobrança mensal/anual para projetistas.

Entretanto:

modelo comercial não deve dirigir a arquitetura neste momento.

Primeiro provar produto.

97. Branding

O nome utilizado é:

Auto AI Builder

Uma identidade visual clean já começou a ser desenvolvida.

Foi escolhida uma opção de logo mais minimalista e posteriormente solicitada em maior definição e mockups.

Isso não afeta a engenharia atualmente.

98. Roadmap histórico

Foi produzido anteriormente um roadmap estruturado com aproximadamente:

15 fases

incluindo gates de validação.

A lógica geral era:

```

Preparacao
|
v
Classificacao
|
v
Modelo semantico
|
v
Validacao
|
v
Pontos
|
v
Builder
|
v
Redes
|
v
Circuitos
|
v
Disciplinas
|
v
IA
|

```

v
Produto

Não assumir que os números antigos das fases ainda representam a prioridade atual.

A descoberta do MCP justifica revisar o roadmap.

99. Estado real do projeto hoje

É importante não confundir visão com implementação.

O Auto AI Builder hoje **não é um produto concluído**.

Existe uma combinação de:

- conhecimento de domínio;
- arquitetura;
- scripts;
- testes;
- protótipos;
- experimentos;
- documentação;
- código desenvolvido no Codex;
- workflows testados;
- regras descobertas.

O ativo principal atualmente é:

conhecimento técnico adquirido + dataset validado + provas de conceito.

100. Maiores riscos técnicos

O maior risco não é “a IA ser inteligente o suficiente”.

Os riscos reais são:

100.1. Confiabilidade geométrica

Converter corretamente coordenadas e orientações.

100.2. Integração Builder

Executar de forma estável.

100.3. Generalização

Algo funcionar em uma planta e falhar completamente em outra.

100.4. Dados

Não existir ground truth suficiente.

100.5. Scope creep

Tentar automatizar elétrico + hidráulico + sanitário + estrutural simultaneamente.

101. Maior vantagem do projeto

Danilo não está criando um software a partir de um problema hipotético.

O projeto nasceu do uso real.

Existe acesso direto a:

- projetos reais;
- DWGs;
- Builder;
- problemas cotidianos;
- validação visual;
- conhecimento de engenharia.

Isso permite um ciclo muito forte:

```

Problema real
|
v
Implementar
|
v
Usar no projeto
|
v
Descobrir erro
|
v
Corrigir
  
```

102. Separação de responsabilidades

102.1. Danilo

Deve ser considerado:

```

Product Owner
+
Domain Expert
+
Acceptance Tester
  
```

Responsável por definir:

- se resultado de engenharia está correto;
- workflow real;
- prioridade;
- regra de negócio;
- comportamento esperado.

102.2. Pablo

Pode assumir progressivamente:

```

Software Architecture
Backend
Desktop Architecture
Integrations
Data Model
  
```

Test Infrastructure
DevOps
Engineering quality

102.3. IA / ChatGPT / Codex

Devem ser tratados como ferramentas de aceleração.

Não como substitutos do ownership técnico.

103. Repositório como Source of Truth

A partir da entrada do Pablo, é fortemente recomendável que o repositório passe a ser a referência oficial.

Chats não devem ser a única fonte das decisões.

Estrutura sugerida:

```
docs/
+-- ADR/
+-- architecture/
+-- domain/
+-- builder/
+-- autocad/
+-- experiments/
```

104. ADR - Architecture Decision Records

Sugestão:

Para decisões importantes criar arquivos como:

```
ADR-001-use-dotnet8.md
ADR-002-use-sqlite.md
ADR-003-semantic-model.md
ADR-004-builder-mcp.md
ADR-005-nondestructive-cad.md
```

Cada ADR:

```
Context
Decision
Alternatives
Consequences
Status
```

Isso evita reabrir a mesma discussão a cada mês.

105. Experimentos também precisam de documentação

Exemplo:

```
EXP-003-builder-outlet-offset.md
```

Conteúdo:

```
Date
Input
Handle
```

Expected
Procedure
Result
Conclusion
Screenshots

O experimento do offset 1,5 / 2,4 cm deveria estar documentado dessa forma.

106. Test data

Não guardar apenas arquivos soltos.

Sugestão:

```
tests/data/project_001/
|
+-- source.dwg
+-- expected_entities.json
+-- expected_associations.json
+-- notes.md
+-- screenshots/
```

107. Antes de Pablo mudar algoritmos

É importante primeiro descobrir:

Qual e atualmente a versao canonica do codigo?

Durante o desenvolvimento existiram:

- V0.3;
- V0.4;
- V0.4.1;
- V0.4.2;
- V0.5;
- V0.6;
- outras tarefas posteriores.

Nem sempre “versão mais nova” significa que todo o conteúdo anterior foi integrado.

108. Auditoria inicial recomendada

Primeiros passos de Pablo no repositório:

1. Build solution
2. Run tests
3. Inventory projects
4. Inventory scripts
5. Find dead code
6. Find duplicate classifiers
7. Find latest schema
8. Find latest mask algorithm
9. Identify fixtures
10. Identify experimental code

Depois criar:

CURRENT_STATE.md

109. Primeiras perguntas que Pablo deve responder

Depois da auditoria:

1. Existe atualmente um domínio central?
2. Onde o DWG é interpretado?
3. Onde são armazenadas associações?
4. Existe SQLite funcionando?
5. Há migrations?
6. Há testes automatizados?
7. O classificador v0.5 está no app ou isolado?
8. Onde vivem os scripts LISP?
9. Existe plugin AutoCAD compilável?
10. Existe interface Builder implementada?
11. Há Computer Use embutido?
12. Existe UI funcional?
13. Existem dados mockados?
14. Onde estão os arquivos de referência?

110. Próximo spike prioritário: MCP

Sugestão de branch:

spike/builder-mcp

Objetivo:

não desenvolver produto; apenas explorar.

Testes:

```
Connect
Read project
Read floors
Read electrical data
Read circuits
Read points
Read properties
Try safe write operation
Read created object
Disconnect
Reconnect
```

Documentar tudo.

111. Critério para substituir UI Automation por MCP

Se o MCP conseguir:

read + create + update + identify objects

de forma estável, tornar MCP a integração primária.

UI Automation passa para:

fallback

ou apenas ações não expostas.

112. Próximo spike: transformação

Criar um laboratório isolado:

TransformationLab

Entradas:

CAD coordinates
Builder coordinates
Known anchors
Rotation

Saída:

Transform matrix
Residual error

Objetivo:

remover calibração manual.

113. Próximo spike: association engine

Outro módulo isolado:

AssociationEngine

Entrada:

point
texts
graphics
geometry

Saída:

associations
score
reasons

Exemplo:

```
Outlet 0x1234
-> "20A" score 0.98
-> "220V" score 0.95
```

114. Explainability

Sempre que possível, o software deve explicar.

Em vez de:

20 A

mostrar:

20 A

Reason:

Text "20A" located 7.2 cm from outlet
No competing outlet within association radius
Orientation compatible
Confidence 98%

Isso aumenta confiança do engenheiro.

115. Não usar IA para mascarar algoritmo ruim

Se o association engine não consegue ligar uma tomada ao texto 20A, primeiro tentar resolver geometricamente.

Não enviar todos os objetos para um LLM e perguntar:

"Qual texto pertence à tomada?"

A IA deve tratar exceções, não substituir modelagem geométrica básica.

116. Estratégia de implantação

Primeiras versões:

```
Developer build
|
v
Danilo test environment
|
v
Real project in assisted mode
|
v
Bug collection
|
v
Regression
```

Não automatizar alterações irreversíveis em projeto real até existir confiança suficiente.

117. Modo Assistido

Seria útil possuir:

Mode = Assisted

onde o programa:

- encontra;
- propõe;
- destaca;
- preenche dados;

mas o usuário confirma.

Depois:

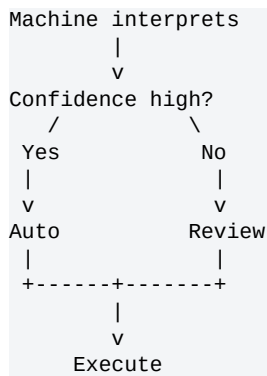
Mode = Automated

para casos de alta confiança.

118. Human in the loop

Este conceito é central.

Workflow:



119. Roadmap revisado recomendado

Fase 0 - Baseline

- auditar repo;
- consolidar documentação;
- congelar datasets.

Fase 1 - Domain

- semantic model;
- SQLite;
- migrations;
- audit.

Fase 2 - CAD

- import;
- classifier;
- associations;
- layers.

Fase 3 - Viewer

- overlay;
- corrections;
- confidence.

Fase 4 - Builder MCP

- exploration;
- abstraction layer.

Fase 5 - Electrical points

- outlet family;

- transform;
- execution;
- validation.

Fase 6 - Electrical expansion

- switches;
- lights;
- data.

Fase 7 - Circuits

- read;
- propose;
- create.

Fase 8 - Conduits

- route proposal;
- execution.

Fase 9 - Hydraulic

Expandir o modelo e integrações para hidráulica.

Fase 10 - Sanitary

Criar dataset, classificador e fluxo equivalente para sanitário.

Fase 11 - AI Assistant

Adicionar assistência contextual quando o core estiver estável.

Fase 12 - Packaging

Instalador, atualização, configuração e diagnóstico.

Fase 13 - Pilot

Uso repetido em projetos reais com métricas.

Fase 14 - Commercialization

Modelo comercial somente após validação robusta do produto.

120. Não priorizar agora

Evitar dispersar tempo neste momento com:

- marketing;
- site;
- estrutura comercial complexa;
- múltiplos modelos de assinatura;
- automação TQS;
- IA multimodelo sofisticada;
- marketplace;
- cloud multi-tenant.

O valor ainda precisa ser provado no vertical slice.

121. Resultado esperado do próximo grande marco

O milestone realmente relevante seria:

Danilo abre uma planta real, o Auto AI Builder reconhece as tomadas, apresenta-as corretamente, recebe aprovação e cria essas tomadas corretamente no Builder de forma repetível.

Se isso funcionar para vários projetos diferentes, o núcleo do produto estará validado.

122. Dados que precisam ser preservados

Nunca perder os datasets históricos que geraram:

```
272 main points
100 associated components
372 semantic objects
```

Nem os testes de:

```
21AF02F
2AFD76
22FAD6F
21AE333
```

Eles contêm aprendizado valioso.

123. Arquivos históricos importantes

Já foram criados/registrados materiais como:

```
AutoAIBuilder_Contexto_Transferencia_Codex.md
Roadmap_Automacao_AutoCAD_AltoQi_Builder.pdf
mascara_camadas_v05.lsp
mascara_limpeza_v06.lsp
```

Esses arquivos devem ser preservados como documentação histórica e referência para a reconstrução do estado atual.

124. Glossário

Máscara

Versão da arquitetura preparada para servir como referência ao projeto complementar.

Ponto

Elemento principal de instalação.

Exemplo:

- tomada;
- chuveiro;
- torneira.

Componente

Elemento complementar associado ao ponto.

Exemplo:

- triângulo;
- texto;
- marcador.

Handle

Identificador de entidade dentro do AutoCAD.

Semantic Layer

Layer cujo nome representa significado técnico, e não simplesmente aparência.

Ground Truth

Dataset manualmente validado usado como referência.

MCP

Protocol/interface atualmente disponível para integração com Builder.

Computer Use

Automação através da interface gráfica.

Offset local

Distância entre referência visual e ponto real de inserção.

Transform global

Conversão entre sistemas de coordenadas.

125. Mental model recomendado para Pablo

A melhor forma de entender o projeto não é:

“Precisamos fazer um robô para mexer no Builder.”

É:

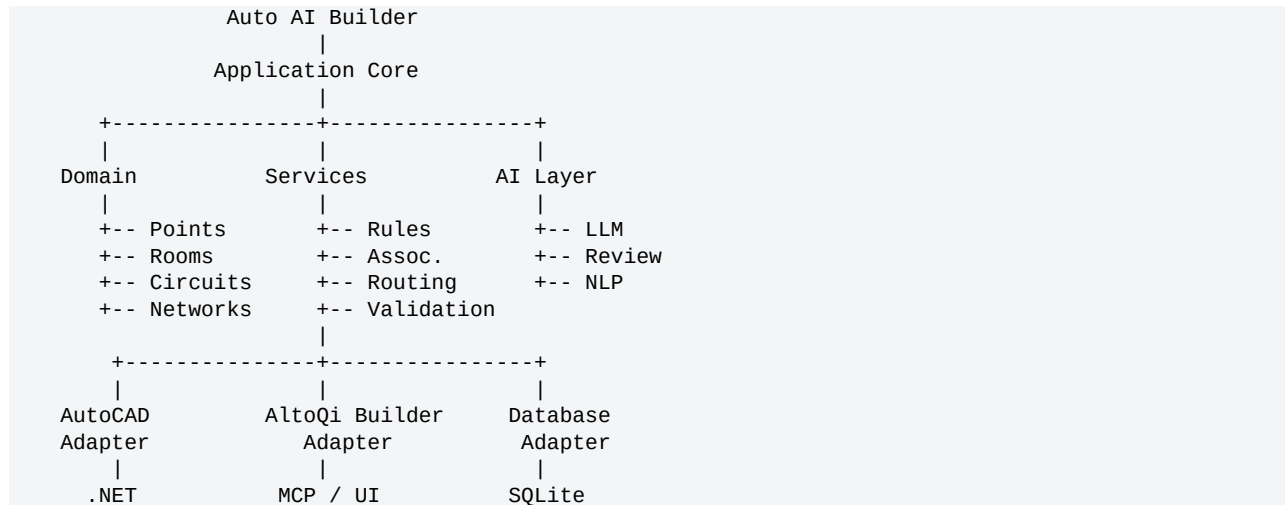
“Precisamos construir um modelo computacional de um projeto complementar e criar adaptadores para ler do AutoCAD e executar no Builder.”

Essa diferença conceitual é fundamental.

O AutoCAD e o Builder são adapters.

O produto é, conceitualmente:

126. Uma arquitetura futura possível



127. Conclusão para Pablo

Pablo,

o Auto AI Builder já passou da fase de “ideia”.

Nós já sabemos que:

- é possível interpretar a máscara;
- é possível reconhecer centenas de pontos;
- é possível estruturar semanticamente os elementos;
- é possível associar informações técnicas;
- é possível preparar o DWG;
- é possível lançar pelo menos determinados tipos de pontos no Builder;
- existem offsets e transforms identificáveis;
- existe um dataset real para regressão;
- existe um caminho arquitetural plausível.

Ao mesmo tempo:

- não existe ainda um produto robusto;
- automação universal do Builder não está resolvida;
- o modelo de domínio precisa ser consolidado;
- a transformação de coordenadas precisa ser formalizada;
- os testes precisam virar infraestrutura;
- a superfície do MCP precisa ser investigada;
- o código histórico precisa ser auditado.

A prioridade agora não é criar mais protótipos desconectados.

A prioridade é transformar todo esse aprendizado em uma **base de software organizada e reproduzível**.

O próximo estágio deve converter:

experimentos
+
scripts

+ chats
+ regras
+ testes manuais

em:

arquitetura
+ modelo de dominio
+ testes de regressao
+ integracoes
+ produto

Esse é o ponto exato de partida.

128. TL;DR técnico para o primeiro dia do Pablo

Depois de ler este documento:

1. Clonar AutoAIBuilder.
2. Compilar tudo.
3. Rodar testes.
4. Inventariar o estado do codigo.
5. Localizar os algoritmos V0.5/V0.6.
6. Localizar o dataset 272 + 100.
7. Formalizar regression tests.
8. Revisar o domain model.
9. Investigar MCP do Builder.
10. Criar um spike de integracao.
11. Formalizar transformacao DWG -> Builder.
12. Fechar um vertical slice completo de tomada.

Não começar por circuitos, condutos ou uma IA sofisticada.

Primeiro provar de forma reproduzível:

DWG
-> compreensao
-> revisao
-> Builder
-> validacao

Quando essa cadeia estiver funcionando de ponta a ponta, o restante da plataforma passa a ser expansão, e não pesquisa fundamental.

Apêndice A - Baselines e contagens históricas

Marco	Contagem / resultado	Observação
Arquivo inicial	~15.400 objetos	Estimativa/contagem inicial do desenho bruto
Triagem preliminar	900 manter / 448 remover / 52 revisar	Snapshot histórico inicial
V0.3.1	1.400 entidades categorizadas	Incluía contexto, mobiliário, descartáveis e revisar
V0.4	272 pontos principais	196 elétricos + 76 hidráulicos
Snapshot consolidado	100 componentes associados	24 ELE_GRAF + 70 ELE_TEXTO

Marco	Contagem / resultado	Observação
		+ 6 HID_GRAF
V0.5	372 objetos semânticos	272 pontos + 100 componentes/textos
V0.5	35 layers semânticas	0 erros de layer
V0.5	0 entidades apagadas / 0 blocos explodidos	Processo não destrutivo
Baixa confiança	~59 objetos	Separados para revisão em uma fase
Piloto de tomadas	~38 tomadas médias	12 direita, 11 cima, 9 esquerda, 6 baixo
Orientações	24 cardinais / 14 inclinadas	Útil para calibração futura

Apêndice B - Casos geométricos conhecidos

Handle	Tipo de caso	Rotação aproximada	Estado
21AF02F	Tomada orientada para baixo	90.02296445°	Validada após ajuste local
2AFD76	Tomada inclinada sudoeste	219.31025869°	Lançamento considerado perfeito após correções
22FAD6F	Caso em revisão	~222.234°	Revisar
21AE333	Caso em revisão	~222.234°	Revisar

Offsets locais observados para tomadas médias:

```

Baixo    -> +1.5 , +2.4
Esquerda -> +2.4 , -1.5
Cima     -> -1.5 , -2.4 [a validar]
Direita  -> -2.4 , +1.5 [a validar]
```

Correção global observada no caso 2AFD76:

```

23,7 cm para a esquerda
25,2 cm para baixo
```

Correção local residual no mesmo caso:

```

1,5 cm para cima
2,4 cm para a esquerda
```

Não hardcodar os valores globais como regra universal.

Apêndice C - Checklist de auditoria inicial do repositório

- ☐ Solution compila em máquina limpa.
- ☐ Testes existentes executam sem intervenção manual.
- ☐ Projetos e assemblies inventariados.
- ☐ Scripts LISP localizados e classificados por versão.
- ☐ Classificadores duplicados identificados.
- ☐ V0.5 e V0.6 localizadas.
- ☐ Dataset 272 + 100 localizado e preservado.
- ☐ Schema SQLite atual identificado.
- ☐ Migrations identificadas ou necessidade delas registrada.
- ☐ Plugin AutoCAD compilável identificado.
- ☐ Código Builder separado entre MCP, UI Automation e protótipos.
- ☐ Código morto e experimental marcado.

- ☐ Dependências externas documentadas.
- ☐ Segredos removidos do repositório.
- ☐ CURRENT_STATE.md criado.

Apêndice D - Checklist do spike MCP

- ☐ Conectar com Builder aberto.
- ☐ Validar comportamento com Builder fechado.
- ☐ Ler projeto atual.
- ☐ Listar pavimentos.
- ☐ Ler dados elétricos.
- ☐ Listar circuitos.
- ☐ Listar pontos.
- ☐ Ler propriedades de um ponto.
- ☐ Descobrir IDs estáveis.
- ☐ Descobrir coordenadas e sistema de referência.
- ☐ Verificar operações de escrita seguras.
- ☐ Criar um objeto de teste, se permitido.
- ☐ Ler novamente o objeto criado.
- ☐ Testar reconexão após reinício do Builder.
- ☐ Documentar autenticação e duração da sessão.
- ☐ Documentar limites e erros.
- ☐ Definir se MCP pode ser adapter primário.

Apêndice E - Estrutura recomendada de documentação no repositório

```
AutoAIBuilder/
|
+-- README.md
+-- PROJECT_HANDOFF.md
+-- CURRENT_STATE.md
+-- ARCHITECTURE.md
+-- ROADMAP.md
+-- CHANGELOG.md
|
+-- docs/
|   +-- ADR/
|   |   +-- ADR-001-use-dotnet8.md
|   |   +-- ADR-002-use-sqlite.md
|   |   +-- ADR-003-semantic-model.md
|   |   +-- ADR-004-builder-mcp.md
|   |   +-- ADR-005-nondestructive-cad.md
|   |
|   +-- experiments/
|   |   +-- EXP-003-builder-outlet-offset.md
|   |
|   +-- architecture/
|   +-- domain/
|   +-- builder/
|   +-- autocad/
|   +-- mcp/
|   +-- computer-use/
|   +-- algorithms/
|   +-- tests/
|
+-- src/
+-- tests/
|   +-- regression/
|   |   +-- mask_v05/
|   +-- data/
|   |   +-- project_001/
```

```
|
|      +-- source.dwg
|      +-- expected_entities.json
|      +-- expected_associations.json
|      +-- notes.md
|      +-- screenshots/
|
+-- prototypes/
+-- samples/
+-- artifacts/
+-- tools/
```

Apêndice F - Arquivos históricos a localizar e preservar

```
AutoAIBuilder_Contexto_Transferencia_Codex.md
Roadmap_Automacao_AutoCAD_AltoQi_Builder.pdf
mascara_camadas_v05.lsp
mascara_limpeza_v06.lsp
```

Além deles, localizar todos os CSVs/JSONs de classificação, screenshots de validação, DWGs usados nos testes e scripts intermediários que tenham sido empregados na construção do ground truth.

Apêndice G - Regra de ouro para o próximo estágio

O próximo estágio do projeto deve priorizar uma única cadeia repetível:

```
DWG
-> semantic model
-> review
-> Builder adapter
-> execution
-> verification
-> audit
```

O projeto deve resistir à tentação de ampliar escopo antes que essa cadeia funcione de forma estável em múltiplos desenhos.

Fim do documento.